



Локальный запуск Telegram-чата

React 19 + Vite 8 + GREEN-API

Назначение документа

Подробная инструкция: от установки Node.js и создания проекта через Vite до запуска чата, настройки Telegram-инстанса, проверки сообщений и production-сборки.

Проект: green-api-chat

Актуально для текущей конфигурации проекта

Версия руководства: 1.0
Дата: 2 октября 2026 года

1. Быстрый запуск

Если проект уже находится на компьютере, для первого запуска достаточно выполнить следующие действия.

1 Откройте терминал

На macOS используйте Terminal, iTerm или встроенный терминал VS Code. На Windows используйте PowerShell или терминал VS Code.

2 Перейдите в папку проекта

Замените путь в примере на фактическое расположение папки **green-api-chat**.

```
cd /путь/к/green-api-chat
```

3 Установите зависимости

Команда прочитает package.json и package-lock.json, затем установит Vite, React и остальные зависимости в папку node_modules.

```
npm install
```

4 Запустите Vite

Команда запускает локальный сервер разработки с автоматическим обновлением страницы после изменения кода.

```
npm run dev
```

После запуска терминал покажет адрес, обычно **http://localhost:5173/**. Откройте его в браузере.

Важно

Не закрывайте терминал, пока работает проект. Для остановки сервера вернитесь в терминал и нажмите Ctrl+C.

Краткая последовательность команд

```
cd /путь/к/green-api-chat  
npm install  
npm run dev
```

2. Подготовка среды разработки

Для работы проекта нужны Node.js, npm, браузер и учётные данные Telegram-инстанса GREEN-API.

2.1. Установка Node.js

Установите актуальную LTS-версию Node.js с сайта nodejs.org. Для Vite 8 рекомендуется современная версия Node.js. Текущая проверенная среда проекта: Node.js 24 и npm 11.

Проверьте установку:

```
node --version  
npm --version
```

Обе команды должны вывести номера версий. Если терминал сообщает, что команда не найдена, перезапустите терминал после установки Node.js.

2.2. Создание нового React-проекта через Vite

Пропустите этот пункт

Если папка `green-api-chat` уже содержит `package.json`, `src` и `vite.config.js`, новый проект создавать не нужно. Перейдите к разделу 3.

Для создания аналогичного пустого проекта React выполните:

```
npm create vite@latest green-api-chat -- --template react  
cd green-api-chat  
npm install  
npm run dev
```

Первая команда создаёт структуру Vite. Параметр **--template react** выбирает React-шаблон. После этого исходный код задания помещается в папку `src`.

2.3. Что уже настроено в проекте

- React 19 отвечает за пользовательский интерфейс и состояние чата.
- Vite 8 запускает dev-сервер и создаёт production-сборку.
- Плагин `@vitejs/plugin-react` подключает обработку JSX и быстрое обновление компонентов.
- Oxlint выполняет статическую проверку JavaScript и React-кода.

3. Установка существующего проекта

После скачивания или клонирования репозитория откройте терминал именно в корневой папке проекта. В ней должен находиться файл `package.json`.

3.1. Проверка текущей папки

macOS и Linux:

```
pwd  
ls
```

Windows PowerShell:

```
Get-Location  
Get-ChildItem
```

В списке должны быть как минимум `package.json`, `package-lock.json`, `vite.config.js`, `index.html` и папка `src`.

3.2. Установка зависимостей

```
npm install
```

Используйте `npm install` при обычной локальной разработке. Для полностью воспроизводимой чистой установки по `package-lock.json` можно использовать:

```
npm ci
```

Не загружайте `node_modules` в Git

Эта папка создаётся локально и может занимать много места. В проекте она уже добавлена в `.gitignore`.

3.3. Основные npm-команды

Команда	Назначение
<code>npm run dev</code>	Запустить Vite в режиме разработки
<code>npm run lint</code>	Проверить исходный код
<code>npm run build</code>	Создать production-сборку в <code>dist</code>
<code>npm run preview</code>	Локально проверить готовую сборку

4. Настройка Telegram-инстанса GREEN-API

Перед входом в приложение подготовьте Telegram-инстанс в личном кабинете GREEN-API. Приложение проверяет состояние и настройки при авторизации.

4.1. Получение учётных данных

- Откройте личный кабинет GREEN-API.
- Выберите Telegram-инстанс.
- Скопируйте значения `idInstance` и `apiTokenInstance`.
- Убедитесь, что состояние инстанса - `authorized`.

Безопасность

Не записывайте `apiTokenInstance` в `App.jsx`, `README`, `.env`, скриншоты или публичный репозиторий. Вводите токен только в форме входа локального приложения.

4.2. Включение входящих сообщений

В личном кабинете GREEN-API выполните следующие шаги:

- Откройте карточку нужного инстанса.
- Нажмите кнопку «Изменить».
- Включите получение уведомлений о входящих сообщениях. Параметр `incomingWebhook` должен иметь значение `yes`.
- Оставьте поле `webhookUrl` пустым. Это необходимо для получения уведомлений через HTTP API методом `ReceiveNotification`.
- Сохраните изменения и дождитесь их применения.

Параметр	Нужное значение	Причина
<code>incomingWebhook</code>	<code>yes</code>	Разрешает входящие уведомления
<code>webhookUrl</code>	пустое поле	Переключает получение на HTTP polling
<code>stateInstance</code>	<code>authorized</code>	Инстанс готов к работе

Если настройка неверна, приложение не откроет чат и покажет инструкцию по исправлению.

5. Запуск и проверка чата

5.1. Запуск Vite

```
npm run dev
```

Пример сообщения терминала:

```
VITE ready  
Local: http://localhost:5173/
```

Откройте Local-адрес в браузере. Если порт 5173 занят, Vite автоматически выберет другой порт - используйте адрес из терминала.

5.2. Вход

- Введите idInstance.
- Введите apiTokenInstance.
- Нажмите «Войти».
- Приложение проверит getStateInstance, getSettings и тип Telegram-инстанса.

5.3. Создание чата

Введите номер получателя в международном формате, используя только цифры. Не добавляйте +, пробелы, скобки или дефисы.

```
77757485939
```

Нажмите «Открыть чат». Приложение вызовет CheckAccount, проверит наличие Telegram-аккаунта и сохранит возвращённый chatId.

5.4. Отправка и получение

- Введите текст в нижнее поле и нажмите «Отправить».
- После успешного SendMessage сообщение появится справа.
- Попросите получателя ответить в Telegram.
- Приложение опрашивает ReceiveNotification и показывает ответ слева.
- После обработки приложение подтверждает уведомление методом DeleteNotification.

Проверка результата

Успешный сценарий: исходящее сообщение отображается справа, ответ получателя появляется слева без обновления страницы.

6. Как работает получение сообщений

Vite обслуживает React-приложение локально, а браузер напрямую выполняет HTTPS-запросы к GREEN-API. Отдельный backend для этого тестового проекта не используется.

Последовательность запросов

Этап	Метод GREEN-API	Результат
Вход	<code>getStateInstance / getSettings</code>	Проверка авторизации и уведомлений
Новый чат	<code>CheckAccount</code>	Получение Telegram chatId
Отправка	<code>SendMessage</code>	Сообщение попадает в чат
Получение	<code>ReceiveNotification</code>	Одно уведомление из очереди
Подтверждение	<code>DeleteNotification</code>	Переход к следующему уведомлению

`ReceiveNotification` использует long polling: запрос может ожидать событие несколько секунд. Пустой ответ означает, что новых уведомлений пока нет. Приложение сразу начинает следующий запрос.

Почему нужен `DeleteNotification`

Очередь работает последовательно. Пока полученное уведомление не подтверждено, следующий вызов может вернуть то же событие. Поэтому приложение обрабатывает уведомление, добавляет сообщение в интерфейс и только затем удаляет событие из очереди.

Ограничение тестового интерфейса

Приложение показывает сообщения активного чата. Уведомления других чатов не отображаются в текущем окне.

7. Проверка и production-сборка

Перед отправкой проекта в репозиторий выполните две проверки.

7.1. Проверка кода

```
npm run lint
```

Команда должна завершиться без ошибок. Исправьте найденные предупреждения перед публикацией.

7.2. Создание сборки

```
npm run build
```

Готовые статические файлы будут созданы в папке dist. Эта папка генерируется автоматически и не должна попадать в Git.

7.3. Проверка сборки

```
npm run preview
```

Откройте адрес preview-сервера из терминала и повторите базовый сценарий входа и открытия чата.

8. Типичные проблемы

Симптом	Что проверить
npm: command not found	Установлен ли Node.js; перезапущен ли терминал
Страница не открывается	Работает ли npm run dev; верен ли Local-адрес
Инстанс не готов	Состояние должно быть authorized
Номер не найден	Формат номера; наличие Telegram-аккаунта
Сообщение не отправляется	Токен, состояние инстанса, ответ API в интерфейсе
Ответ не появляется	incomingWebhook=yes; webhookUrl пуст; получатель ответил в нужный чат
Порт занят	Используйте новый порт, показанный Vite, или остановите другой процесс

9. Финальный контрольный список

Перед демонстрацией проекта проверьте каждый пункт:

- Node.js и npm доступны в терминале.
- npm install завершился без ошибок.
- npm run dev запустил Vite и показал Local-адрес.
- Инстанс GREEN-API имеет тип telegram и состояние authorized.
- В настройках через кнопку «Изменить» установлено incomingWebhook=yes.
- Поле webhookUrl оставлено пустым.
- Вход с idInstance и apiTokenInstance выполнен успешно.
- CheckAccount находит получателя по международному номеру.
- Текстовое сообщение успешно отправляется.
- Ответ получателя появляется в интерфейсе без перезагрузки.
- npm run lint и npm run build завершаются успешно.
- Учётные данные не сохранены в исходном коде и Git.

Минимальный набор команд

```
cd /путь/к/green-api-chat
```

```
npm install
```

```
npm run dev
```

Проверка перед сдачей:

```
npm run lint
```

```
npm run build
```

После успешного выполнения этих шагов проект готов к локальной демонстрации.